

ImageStream e ImageStreamTag

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Spiegare il ruolo degli **ImageStream** come livello di astrazione sui riferimenti alle immagini container
- Creare e gestire **ImageStream** e **ImageStreamTag** con **oc** e manifest YAML
- Configurare il polling automatico delle immagini con **importPolicy.scheduled: true**
- Usare **ImageStreamImage** per fare riferimento immutabile a un digest sha256
- Configurare i trigger **ImageChange** su **DeploymentConfig** e **BuildConfig**
- Abilitare la **lookupPolicy** locale per usare nomi brevi nei Deployment
- Integrare ImageStream con Azure Container Registry (ACR) su ARO

Teoria e Concetti Chiave

Cos'è un ImageStream

Un **ImageStream** è un oggetto OpenShift che fornisce un livello di **astrazione e indirection** sui riferimenti alle immagini container. Invece di fare riferimento direttamente a **registry.access.redhat.com/ubi9/ubi-minimal:9.4**, si usa **ubi-minimal:9.4** (nome breve) e l'ImageStream risolve il riferimento reale.

Vantaggi degli ImageStream:

Caratteristica	Senza ImageStream	Con ImageStream
Riferimento immagine nei manifest	URL completo del registry	Nome breve del namespace
Aggiornamento immagine	Modifica manuale di tutti i Deployment	Trigger automatico
Portabilità	Dipende dall'URL del registry	Indipendente dal registry
Tracciamento della storia	Non disponibile	status.tags[].items[]
Rollback	Manuale	oc rollout undo con trigger

Struttura di un ImageStream

Un **ImageStream** contiene:

- **spec.lookupPolicy.local**: se **true**, i Pod nel namespace possono usare il nome breve dell'ImageStream invece dell'URL completo del registry
- **spec.tags[]**: lista dei tag con il riferimento sorgente (DockerImage, ImageStreamTag, ImageStreamImage)
- **status.dockerImageRepository**: URL del registry interno dove le immagini dell'IS sono ospitate
- **status.tags[]**: lista dei tag risolti con storia delle versioni (ogni import crea un nuovo item)

ImageStreamTag (ISTag)

Un **ImageStreamTag** è un oggetto che rappresenta un singolo tag di un ImageStream. È referenziato nel formato:

```
<imagestream-name>:<tag>
```

o come oggetto separato con **kind: ImageStreamTag**.

from.kind nei tag di un ImageStream:

Kind	Descrizione	Esempio
DockerImage	Punta a un'immagine su un registry esterno o interno	registry.access.redhat.com/ubi9/ubi-minimal:9.4
ImageStreamTag	Punta a un tag di un altro ImageStream (o stesso IS)	myapp:stable → myapp:latest
ImageStreamImage	Punta a un'immagine tramite sha256 digest (immutabile)	myapp@sha256:abc123...

ImageStreamImage (ISImage)

Un **ImageStreamImage** è un riferimento **immutabile** a un'immagine tramite il suo digest sha256. Il formato è:

```
<imagestream-name>@sha256:<digest>
```

A differenza di un tag (che può essere spostato), un digest identifica sempre la stessa layer di immagine.

Import Automatico con `importPolicy.scheduled`

Se `spec.tags[].importPolicy.scheduled: true`, OpenShift interroga periodicamente il registry sorgente per vedere se il tag è stato aggiornato. Se l'immagine è cambiata, l'ImageStreamTag viene aggiornato e vengono attivati i trigger configurati.

 **Suggerimento:** Usare `scheduled: true` per i tag di immagini base (es. `ubi-minimal:latest`) in modo da ricevere automaticamente patch di sicurezza.

Trigger ImageChange

Il **trigger ImageChange** fa sì che una nuova build o un nuovo deployment venga avviato automaticamente quando l'immagine referenziata in un ImageStreamTag viene aggiornata.

Trigger	Oggetto	Effetto
<code>ImageChangeTrigger</code> in <code>BuildConfig</code>	BuildConfig	Avvia una nuova build
<code>ImageChangeTrigger</code> in <code>DeploymentConfig</code>	DeploymentConfig	Avvia un nuovo deployment
<code>--from-image</code> in <code>oc set triggers</code>	Deployment (non DC)	Aggiunge trigger su Deployment standard

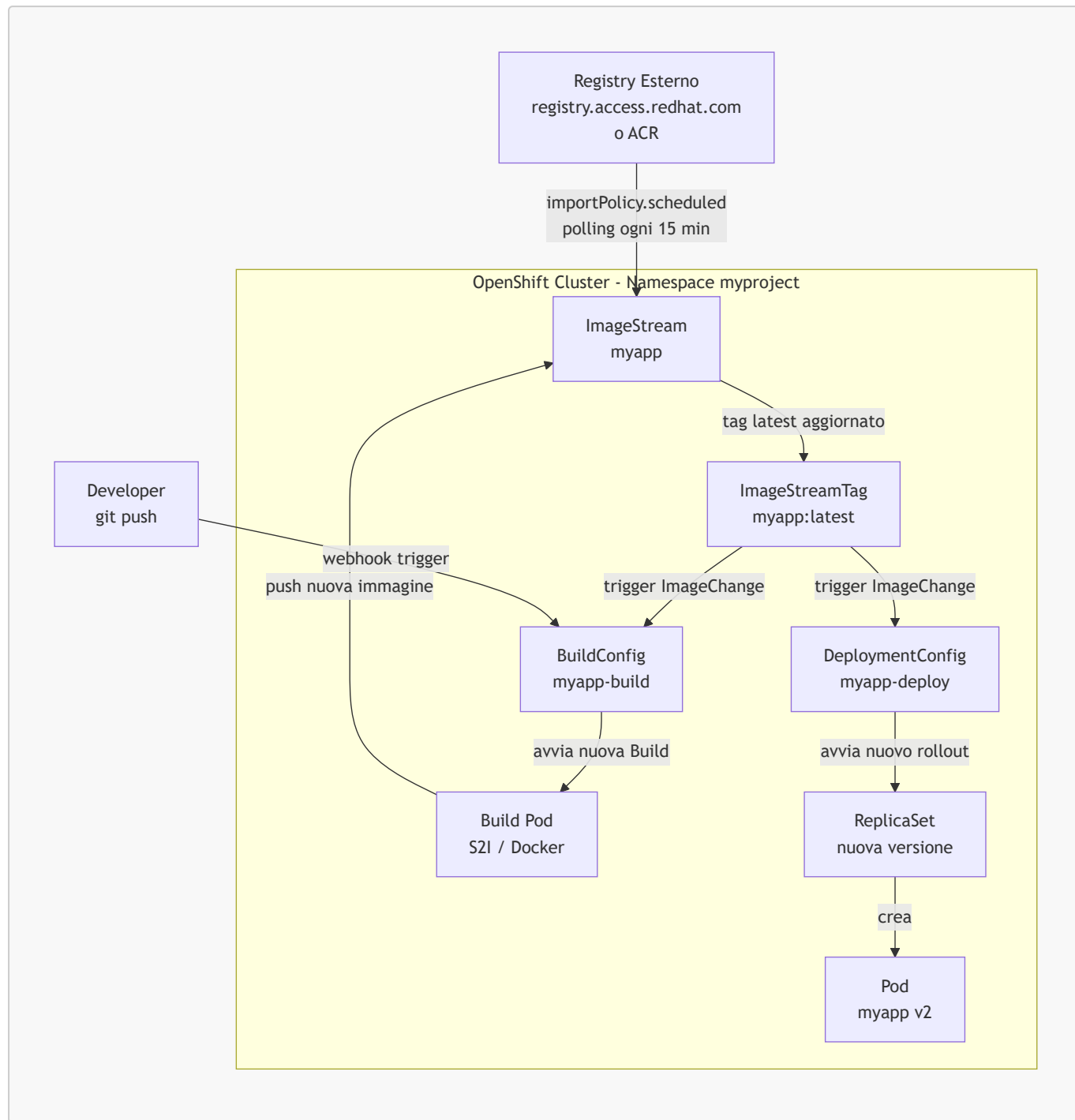
Lookup Policy Locale

Con `spec.lookupPolicy.local: true`, i Pod nel namespace possono fare riferimento all'immagine con il solo nome breve (es. `myapp:latest`) invece dell'URL completo del registry. OpenShift intercetta la richiesta e la risolve all'ImageStream.

 **Attenzione:** La lookup policy locale richiede che il nodo abbia la corretta configurazione CRI-O. Non funziona se il Pod usa un ServiceAccount senza permesso di pull dall'IS.

Diagrammi

Ciclo ImageStream → Trigger → Deployment



Comandi Pratici con Output Atteso

```
# Creare un ImageStream vuoto (senza tag)
oc create imagestream myapp -n myproject

# Output atteso:
# imagestream.image.openshift.io/myapp created
```

```
# Importare un'immagine esterna in un ImageStream esistente
oc import-image myapp:latest \
  --from=registry.access.redhat.com/ubi9/ubi-minimal:9.4 \
  --confirm \
  -n myproject

# Output atteso:
# imagestream.image.openshift.io/myapp imported
#
# Name:                myapp
# Namespace:           myproject
# Created:              Less than a second ago
# Labels:               <none>
# Docker Pull Spec:    image-registry.openshift-image-
registry.svc:5000/myproject/myapp
# Image Lookup:         local=false
# Unique Images:        1
# Tags:                 1
#
# latest
#   tagged from registry.access.redhat.com/ubi9/ubi-minimal:9.4
#   * registry.access.redhat.com/ubi9/ubi-minimal@sha256:a1b2c3...
#   Less than a second ago
```

```
# Taggare un'immagine aggiuntiva nell'ImageStream
oc tag registry.access.redhat.com/ubi9/ubi-minimal:9.4 \
  myapp:stable \
  -n myproject

# Output atteso:
# tag.image.openshift.io/myapp:stable set to registry.access.redhat.com/ubi9/ubi-
minimal:9.4
```

```
# Descrivere un ImageStream con tutti i tag e la storia delle importazioni
oc describe imagestream myapp -n myproject

# Output atteso:
```

```
# Name:                myapp
# Namespace:           myproject
# Created:              2 minutes ago
# Tags:                latest, stable
#
# latest
#   tagged from registry.access.redhat.com/ubi9/ubi-minimal:9.4
#   * registry.access.redhat.com/ubi9/ubi-minimal@sha256:a1b2c3...
#     2 minutes ago
#
# stable
#   tagged from registry.access.redhat.com/ubi9/ubi-minimal:9.4
#   * registry.access.redhat.com/ubi9/ubi-minimal@sha256:a1b2c3...
#     1 minute ago
```

Elencare tutti gli ImageStreamTag di un namespace
`oc get imagestreamtag -n myproject`

```
# Output atteso:
# NAME                IMAGE REF
UPDATED
# myapp:latest        registry.access.redhat.com/ubi9/ubi-minimal@sha256:a1b2c3...
2m
# myapp:stable         registry.access.redhat.com/ubi9/ubi-minimal@sha256:a1b2c3...
1m
```

Abilitare il polling automatico su un tag
`oc tag registry.access.redhat.com/ubi9/ubi-minimal:9.4 \`
`myapp:latest \`
`--scheduled \`
`-n myproject`

```
# Output atteso:
# tag.image.openshift.io/myapp:latest set to registry.access.redhat.com/ubi9/ubi-
minimal:9.4 (scheduled)
```

Aggiungere un trigger ImageChange a un Deployment standard
`oc set triggers deployment/myapp-deploy \`
`--from-image=myapp:latest \`
`--containers=myapp \`
`-n myproject`

```
# Output atteso:
# deployment.apps/myapp-deploy triggers updated
```

```
# Abilitare la lookupPolicy locale su un ImageStream
oc set image-lookup imagestream/myapp -n myproject

# Output atteso:
# imagestream.image.openshift.io/myapp image lookup updated
```

```
# Verificare la lookupPolicy locale
oc get imagestream myapp -n myproject \
  -o jsonpath='{.spec.lookupPolicy.local}'

# Output atteso:
# true
```

```
# Riferirsi a un'immagine tramite digest (ImageStreamImage)
oc get imagestreamimage "myapp@sha256:a1b2c3d4e5f6" -n myproject \
  -o jsonpath='{.image.dockerImageMetadata.Config.Cmd}'

# Output atteso:
# ["/bin/bash"]
```

Esempi YAML Completi

ImageStream Completo con Tag latest (DockerImage) e stable (sha256 digest)

```
# ImageStream: oggetto OpenShift che astrae i riferimenti alle immagini container
apiVersion: image.openshift.io/v1
kind: ImageStream
metadata:
  name: myapp
  namespace: myproject
  labels:
    app: myapp
spec:
  # Lookup policy locale: true = i Pod del namespace possono usare il nome breve
  "myapp:tag"
  lookupPolicy:
    local: true # ← abilita risoluzione automatica del
    nome breve

  tags:
    # Tag "latest": punta a un'immagine esterna con polling automatico
    - name: latest
      from:
        kind: DockerImage # ← tipo di sorgente: immagine su
        registry esterno
        name: registry.access.redhat.com/ubi9/ubi-minimal:9.4 # ← URL completo
        dell'immagine
        importPolicy:
          scheduled: true # ← OpenShift esegue polling periodico
          (ogni ~15 min)
          importMode: PreserveOriginal # ← PreserveOriginal | Legacy
          referencePolicy:
            type: Source # ← Source = pull diretto dall'URL
            sorgente # ← Local = pull dal registry interno
            (cache)

    # Tag "stable": punta a un digest sha256 specifico (riferimento immutabile)
    - name: stable
      from:
        kind: DockerImage
        name: registry.access.redhat.com/ubi9/ubi-
        minimal@sha256:a1b2c3d4e5f6789abcdef0123456789abcdef0123456789abcdef0123456789a
        importPolicy:
          scheduled: false # ← digest non cambia, non serve
          polling
          referencePolicy:
            type: Local # ← serve l'immagine dalla cache
            interna

    # Tag "acr-prod": punta a un'immagine su Azure Container Registry
```



```
- name: acr-prod
  from:
    kind: DockerImage
    name: myaroregistry.azurecr.io/myapp:production # ← immagine su ACR
  importPolicy:
    scheduled: true
    importMode: PreserveOriginal
  referencePolicy:
    type: Local
```

Note Specifiche per Azure ARO

Uso degli ImageStream con Azure Container Registry (ACR)

Su ARO, è comune usare un ImageStream che punta a immagini su ACR. Per permettere a OpenShift di importare automaticamente le immagini, è necessario fornire le credenziali ACR:

```
# Ottenere le credenziali di accesso all'ACR
ACR_NAME=myaroregistry
ACR_USER=$(az acr credential show --name ${ACR_NAME} --query username -o tsv)
ACR_PASS=$(az acr credential show --name ${ACR_NAME} --query "passwords[0].value" -o tsv)

# Creare un Secret dockerconfigjson con le credenziali ACR
oc create secret docker-registry acr-pull-secret \
  --docker-server=${ACR_NAME}.azurecr.io \
  --docker-username=${ACR_USER} \
  --docker-password=${ACR_PASS} \
  -n myproject

# Associare il Secret al ServiceAccount "default" per il pull
oc secrets link default acr-pull-secret --for=pull -n myproject

# Importare un'immagine da ACR usando il Secret
oc import-image myapp:acr-prod \
  --from=${ACR_NAME}.azurecr.io/myapp:production \
  --confirm \
  --reference-policy=Local \
  -n myproject
```

 **Suggerimento:** Per evitare di usare credenziali statiche ACR, configurare la Managed Identity del cluster ARO con il ruolo `AcrPull` sull'ACR. In questo modo il pull avviene tramite l'identità gestita e non richiede un Secret (vedi file 05).

Trigger ImageChange con ACR su ARO

Quando il tag di un'immagine su ACR viene aggiornato, OpenShift può rilevare il cambiamento tramite il polling automatico (`scheduled: true`) e avviare un nuovo deployment:

```
# Configurare il polling automatico per un tag ACR
oc tag myaroregistry.azurecr.io/myapp:production \
  myapp:production \
  --scheduled \
  -n myproject

# Verificare che il polling sia attivo
oc get imagestreamtag myapp:production -n myproject \
  -o jsonpath='{.tag.importPolicy.scheduled}'

# Output atteso:
# true
```

Riepilogo dei Punti Chiave

- Un `ImageStream` è un livello di astrazione sui riferimenti alle immagini: separa i manifest applicativi dall'URL fisico del registry
 - I tag di un IS possono puntare a `DockerImage` (URL registry), `ImageStreamTag` (altro tag), o `ImageStreamImage` (digest sha256 immutabile)
 - `importPolicy.scheduled: true` abilita il polling automatico del registry sorgente; quando l'immagine cambia, i trigger vengono attivati
 - Il trigger `ImageChange` su `BuildConfig` o `DeploymentConfig` avvia automaticamente build/deployment alla variazione del tag
 - `spec.lookupPolicy.local: true` permette ai Pod del namespace di usare il nome breve (`myapp:latest`) invece dell'URL completo del registry
 - I comandi principali sono: `oc import-image`, `oc tag`, `oc describe imagestream`, `oc get imagestreamtag`, `oc set triggers`, `oc set image-lookup`
 - Su ARO, gli ImageStream si integrano con ACR tramite Secret `dockerconfigjson` o Managed Identity
-

Domande di Verifica

1. Qual è la differenza principale tra `ImageStreamTag` e `ImageStreamImage`?

- A) `ImageStreamTag` è immutabile, `ImageStreamImage` può cambiare
- B) `ImageStreamTag` punta a un tag (mutabile), `ImageStreamImage` punta a un digest sha256 (immutabile) ☒
- C) `ImageStreamTag` è per immagini interne, `ImageStreamImage` per immagini esterne
- D) Non c'è differenza, sono sinonimi

2. Quale campo di un `ImageStream` abilita il polling periodico dell'immagine sorgente?

- A) `spec.tags[].importPolicy.scheduled: true` ☒
- B) `spec.lookupPolicy.local: true`
- C) `spec.tags[].referencePolicy.type: Local`
- D) `spec.autoUpdate: true`

3. Cosa fa `spec.lookupPolicy.local: true` su un `ImageStream`?

- A) Importa automaticamente l'immagine dal registry locale
- B) Permette ai Pod del namespace di usare il nome breve dell'`ImageStream` senza specificare l'URL completo del registry ☒
- C) Blocca il pull delle immagini da registry esterni
- D) Abilita il caching delle immagini nel registry interno

4. Quale comando aggiunge un trigger `ImageChange` a un `Deployment` standard (non `DeploymentConfig`)?

- A) `oc patch deployment/myapp --type=merge --patch='{ "spec": { "triggers": [...] } }'`
- B) `oc set triggers deployment/myapp --from-image=myapp:latest --containers=myapp` ☒
- C) `oc create trigger imagechange --deployment=myapp --image=myapp:latest`
- D) `oc annotate deployment/myapp openshift.io/image-trigger=myapp:latest`

5. Quando si usa `from.kind: ImageStreamTag` nella definizione di un tag di un `ImageStream`, cosa si ottiene?

- A) Un riferimento a un'immagine su un registry esterno tramite URL
- B) Un riferimento immutabile tramite digest sha256
- C) Un tag che punta a un altro tag dello stesso o di un altro `ImageStream` (aliasing tra tag) ☒
- D) Un riferimento a un'immagine nel registry interno senza passare per il DNS del cluster